Question 1: Functional List Operations

All code for Question 1 is in `Question1.hs`. The file contains one `main` method that tests every function with small examples.

(a) **Sum and multiply by 5**

`sumTimesFive` first calculates the sum of the input list and then multiplies the result by 5.

Example:

```
sumTimesFive [1,2,3,4] = 50
```

(b) **Split even and odd indices**

`splitEvenOddIndices` treats the first element as index 0, so it goes into the even-index list. The function recursively consumes two elements at a time. The first element goes to the even list and the second element goes to the odd list.

Example:

```
splitEvenOddIndices [1,2,3,4,5] = ([1,3,5],[2,4])
```

(c) **Reverse the middle of a string**

`reverseMiddle` leaves the first and last characters in place and reverses only the characters between them. Empty strings and one-character strings stay unchanged.

Example:

```
reverseMiddle "Haskell" = "Hleksal"
```

(d) **Perfect square**

`isPerfectSquare` checks only positive integers. It calculates the integer part of the square root and then squares that value again. If the squared result is the original number, then the number is a perfect square.

Examples:

```
isPerfectSquare 16 = True
isPerfectSquare 20 = False
```

(e) **Count even numbers**

`countEvenNumbers` uses a list comprehension to select only the even values and then counts how many values remain.

Example:

```
countEvenNumbers [1..10] = 5
```

(f) **GCD of three integers**

`gcdThree` applies `gcd` step by step. First it finds the GCD of the first two numbers, then it finds the GCD of that result and the third number.

Example:

`gcdThree 24 60 96 = 12`

(g) **Perfect numbers below 10000**

`properDivisors` finds all positive divisors smaller than the number itself. `perfectNumbersBelow10000` keeps the numbers that are equal to the sum of their proper divisors.

Result:

`[6,28,496,8128]`

Question 2: Simple Infinite Lists

All code for Question 2 is in `Question2.hs`. Since the lists are infinite, the `main` method only prints finite prefixes with `take`.

(a) **Infinite list of multiples of 3**

The list is:

```
multiplesOfThree = [3, 6 ..]
```

The first 15 values are:

`[3,6,9,12,15,18,21,24,27,30,33,36,39,42,45]`

Lazy evaluation is useful because Haskell does not compute the whole infinite list at once. It only computes as many elements as the program asks for. For example, `take 15 multiplesOfThree` only demands the first 15 elements.

(b) **Infinite list of natural numbers and squares**

The list is:

```
numberSquarePairs = [(n, n * n) | n <- [1 ..]]
```

The first 10 pairs are:

`[(1,1),(2,4),(3,9),(4,16),(5,25),(6,36),(7,49),(8,64),(9,81),(10,100)]`

(c) **Numbers divisible by both 3 and 5**

Numbers divisible by both 3 and 5 are multiples of 15.

The list is:

```
divisibleByThreeAndFive = [15, 30 ..]
```

The first 15 values are:

`[15,30,45,60,75,90,105,120,135,150,165,180,195,210,225]`

Q3

See Question3.hs

Q4

See Question4.hs

Q5

See Question5.hs

Q6

(a) Currying is the rewriting of a function of multiple inputs as a sequence of functions each taking a single input.

$$f(x, y, z) \quad \text{can be rewritten as} \quad f(x)(y)(z)$$

I.e. a function of type

$$A \times B \to C$$

becomes a function of type

$$A \to B \to C.$$

Uncurried:

```
add(x,y) = x + y
```

Curried:

```
add(x)(y) = x + y
```

There is a natural correspondence

$$A \times B \to C \cong A \to (B \to C),$$

so in principle any function of multiple arguments can be rewritten in curried form (and vice versa), although in practice this may require some effort depending on the language.

Difference to tupled functions A function taking a tuple has type

$$A \times B \to C$$

and receives both arguments at once, whereas a curried function

$$A \to B \to C$$

takes its arguments one after another and returns intermediate functions.

Benefits Currying allows:

- **Partial application:** fixing some arguments to obtain new functions, e.g.

  `add(5)`

  gives a function $y \mapsto 5 + y$.
- **Code reuse:** specialized functions can easily be derived from general ones.
- **Abstraction:** functions can be composed and passed around more flexibly.
- **Cleaner functional style:** especially useful in functional languages like Haskell.

(b) The given Haskell code

```
scale :: Double -> Double -> Double
scale factor = \x -> factor * x
```

defines a curried function `scale` that takes a number `factor` and returns a new function. This returned function takes an argument `x` and multiplies it by `factor`.

Thus, `scale` has type

$$\texttt{Double} \to (\texttt{Double} \to \texttt{Double}),$$

which is a curried function.

It also demonstrates **partial application**:

```
double = scale 2
```

Here, `scale` is applied to a single argument `2`. The result is a new function

$$\texttt{double} = \lambda x.\, 2 \cdot x,$$

which doubles its input.

Evaluating, for example,

```
double 3
```

yields

$$2 \cdot 3 = 6.$$

(c) Signature:

```
Double -> Double -> Double
```

Code in Question6.hs

Q7

The decrypted version reads:

Professor: “You are late in submitting your Haskell assignment”. Student: “In fact, I have submitted it, but you have not started evaluating it yet”. Professor: “Actually, I have, but I have not been asked to submit the grades yet”!

Clearly it is about Lazy eval, which is the prctice of evaluating if and only if a specific value is needed.